

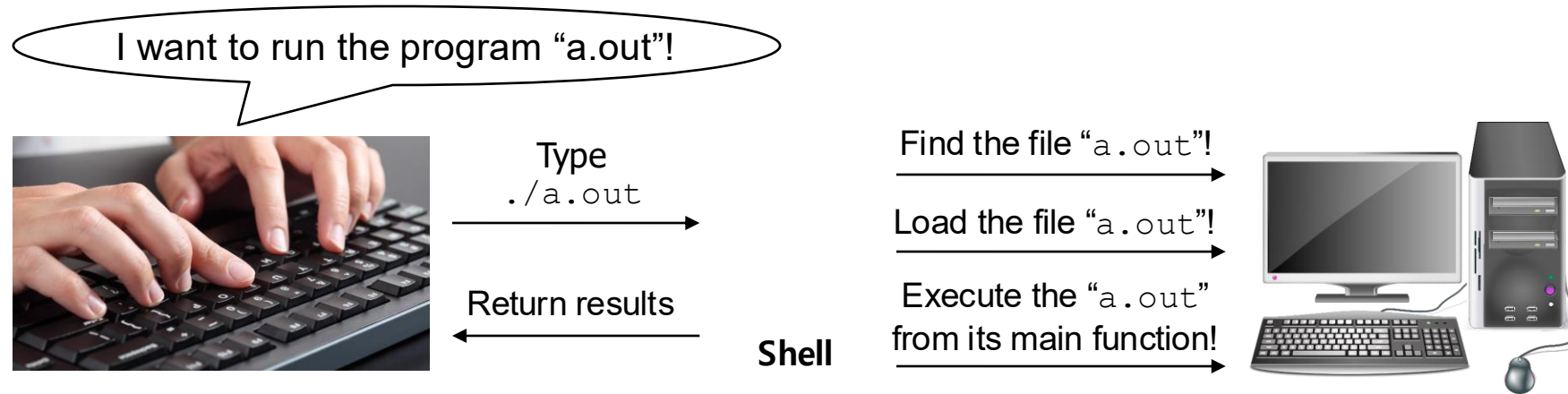
Lab 4. A Unix Shell

System Programming Assignment

Seungjae Lee
SNU Secure and Reliable Systems Lab.

What is a shell?

- Shell: a Program that executes commands typed by a user



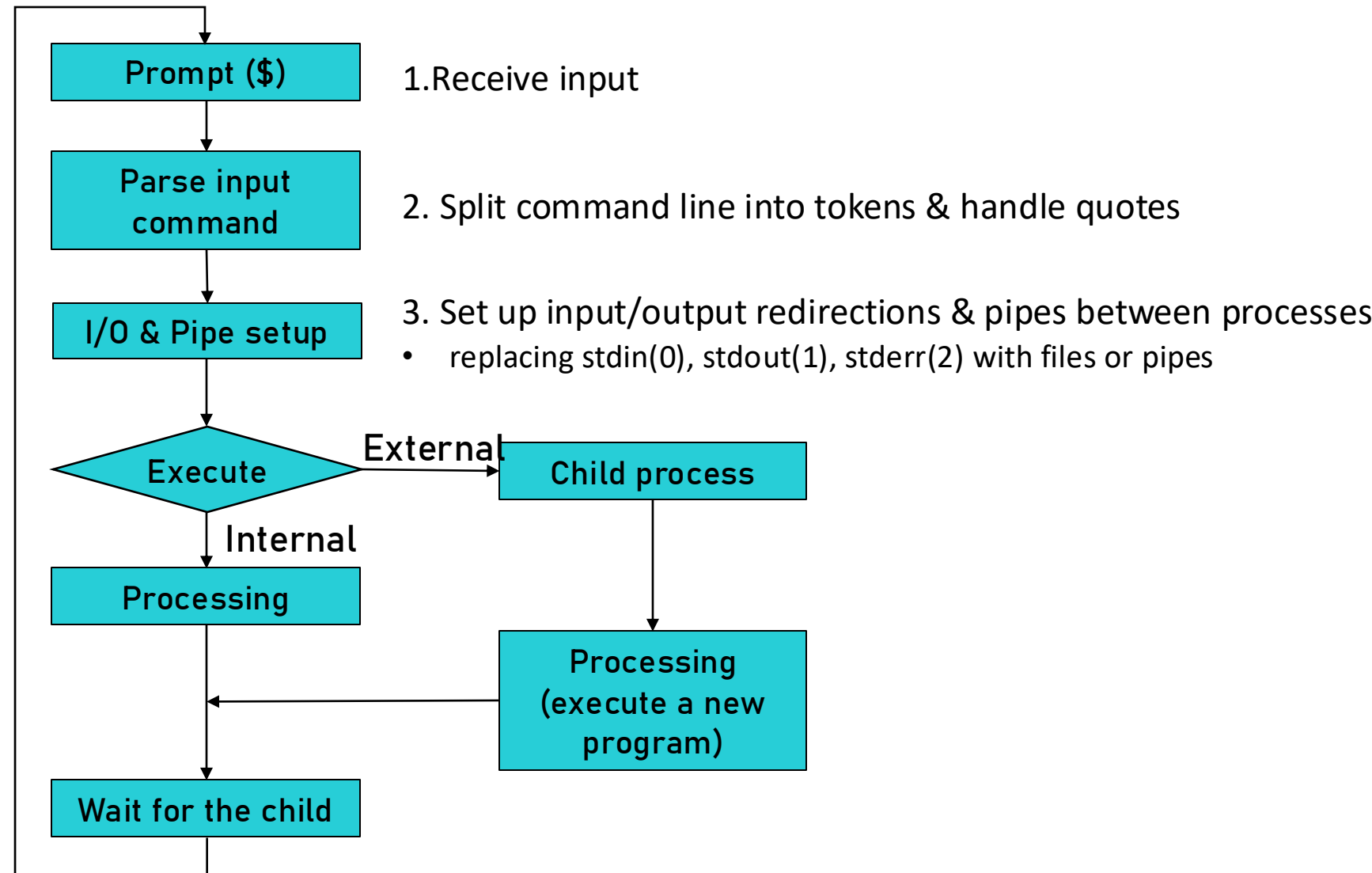
A shell =

```
while(1) {
```

- Receives an input command string from the user
- Interprets the command
- Executes the command requested by user

```
}
```

Basic Shell Flow



4. Execute 2 types of command

- Built-in: cd, exit, export
 - Executed directly by the shell itself w/o creating a new process
- External: ls, grep
 - Separate executable program
 - Create a new process and execute with fork()->exec*()

Let's make a simple shell - SNUSH

Goal: Understand Unix processes, low-level I/O, and signal handling

1. Implement basic shell functionality

- Utilize parsed command line
- Add job(s) to a job manager
- External command execution

2. Support for redirection (<, >)

- Standard input redirection
- Standard output redirection

3. Support for pipe (|)

- Single pipe support
- Multiple pipe support

sh(Bourne Shell)

ksh(Korn Shell)

chs(C shell)

tcsh

bash(Bourne Again Shell)

snush(SNU Shell)

Extra Credit for Background Process Support

- 4. Support for background process
 - Running a process in the background mode
 - Handle the process when the background process terminates

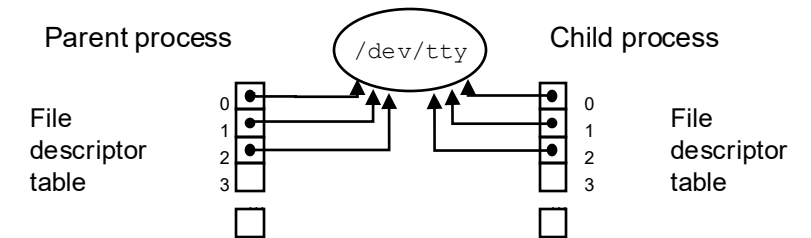
Basic Shell Functions

APIs Behind Shell Execution

- Why do we have to create a new child process for an external command?
- OS runs `exec()` family of system calls to run a program
 - Replace code and data entirely
- If executed directly on the shell, the shell process itself would be replaced and lost

APIs Behind Shell Execution

- `fork()` creates a child process that duplicates the parent
 - Duplicate parent's file descriptor table
 - Changes to FD numbers(e.g., close/dup) in one process don't affect the other
 - The open file descriptions that the FD entries point to are shared.



- `execvp(const char *__file, char const __argv[])`
 - Automatically searches directory in `$PATH` (env. var.) to find executable binary if it contains no slashes in `file`.
 - EX) "ls" instead of "/usr/bin/ls"
 - Replace the current process image with a new program

A Minimal Shell for Executing a Single Command

- Simple shell code
 - Job = user's command line input
 - ex) "ps -ef | grep job"
 - One job, Two processes.
- Notice
 - Return value/error checking is ***mandatory*** when doing the assignment to ensure proper error handling
 - The code here is simplified just for easy understanding

```
fgets(command, sizeof(command), stdin);
command[strlen(command)-1] = 0;
```

① Get the command

```
token = strtok(command, " ");
if (token == NULL) {exit(-1);}
```

② Parse(Tokenize) command

```
arguments[0] = token;
for (i = 1; i < 10; i++) {
    token = strtok(NULL, " ");
    if (token == NULL)
        break;
    arguments[i] = token;
}
arguments[i] = NULL;
```

③ Compose argument array

```
pid = fork();
```

④ Fork child process

```
if (pid == 0){
    execvp(arguments[0], arguments);
}
```

⑤ Child process:
changes its execution
image to arguments[0]

```
/* Parent process */
while(1) {
    // Check if a foreground job is
    // still running
}
```

⑥ Parent process :
while loop waiting for
the foreground job to finish

Your TO-DOs for snush: Basic Shell Functions

Functions for Basic Shell

- **Implement external command execution using `fork()` and `execvp()`.**
- No need to implement internal (built-in) commands!
 - Two built-in commands are provided
 - `cd` command
 - Moves to the specified directory
 - If no argument is provided, use the env. variable (`HOME`) to navigate to the home directory
 - `exit` command to exit the shell `snush`
- **Implement `fork_exec()`, `handle_sigchld()`, `handle_sigint()`**
- Important `snush` functions to utilize
 - `build_command()`: a wrapper for `build_command_partial()`
 - Constructs the argument array before command execution.
 - Uses a full range of token indices
 - `wait_fg()`: continuously checks whether the given job has finished executing

Functions for Basic Shell

- You need to implement `fork_exec()`, `handle_sigchld()`, `handle_sigint()`
- Important system calls to utilize
 - `fork()`, `dup2()`, `execvp()`
 - `pipe()`, `open()`, `close()`
 - `setpgid()`
 - Sets the process group ID(PGID) of a process
 - Child creates a new PGID, so it can be managed independently for job control and signal handling (otherwise parent(shell) shell might also be affected)
 - `tcsetpgrp()` = terminal control set process group
 - Assigns terminal control to a specific process group.
 - The shell uses it to give control to the foreground job while it runs and then restore control back to itself after the job finishes.
 - Ensures that terminal input signals (e.g., Ctrl+C) are delivered to the correct process group (the running job, not the shell).

Implementing Basic Shell Functions - `fork_exec()`

Handle **single external command + I/O redirection**

1. Create a child process
2. Convert the tokenized input into an argument array for an external command
3. Do appropriate actions for supporting I/O redirection
4. Parent process should add job to a job manager
5. Execute the command in the child process
6. Make sure the parent process waits properly with `wait_fg()`
7. The return value of `fork_exec()` should be the job id

Job = The user's command line input

- * Needs jobs & job manager to track, control, signal groups of processes.
- * Allowing features like foreground control, signal control, status track..

Job Manager

- Registers each job
- Assign unique job id
- Tracks all processes belong to the job

Implementing Basic Shell Functions - `handle_sigchld()`

- ① When a child process exits:
 - Parents process receives `SIGCHLD` signal
- ② Parent process calls `waitpid()` to check the child's termination
- ③ Job manager cleanup
 - The exited process's `PID` is removed from the `PID list` of its corresponding job.
 - If all processes in the job have exited, the `entire job entry is removed` from the job manager.

Implementing Basic Shell Functions - `handle_sigint()`

- ① When the user presses Ctrl+C, it should:
 - Interrupt the foreground job and sends signals to all processes
 - Keep the shell itself running

Support for Redirection

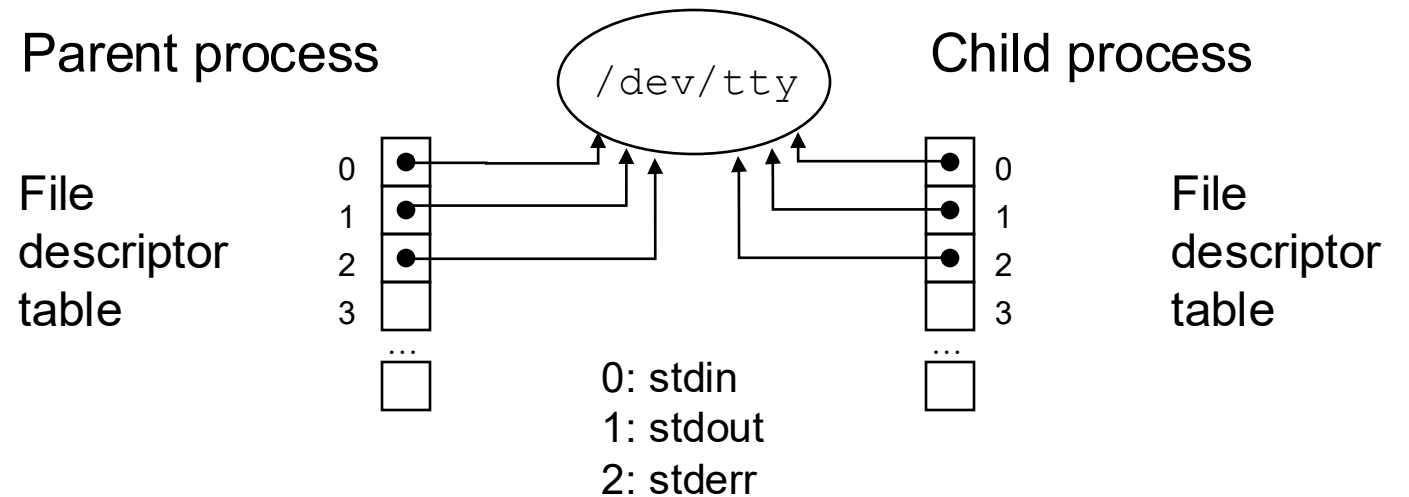
System Calls for Redirection

- Shell can change the `stdin`, `stdout`, `stderr` of a command to/from a file instead of terminal
 - `Stdin(<)` : standard input.
 - `Stdout(>)` : standard output
- How?
 - Use `open()` or `create()` to open a file for reading or writing, creating the file
 - Use `close()` to close unused file descriptors
 - Use `dup2()` to duplicate the file descriptor to any target descriptor, including `stdin`, `stdout` or `stderr`
- **For this assignment, support redirection of only `stdin/stdout`**
 - No `stderr(2>)`, `stdout append(>>)`

Simple Redirection Trace (1)

- Simple code for implementing stdout redirection. "someprogram > somefile"
- A new child has the identical file descriptor table as parent's

```
pid = fork();  
if (pid == 0){  
    /* Child process */  
    fd = creat("somefile", 0640);  
    dup2(fd, 1);  
    close(fd);  
    execvp("someprogram", program_args);  
    fprintf(stderr, "exec failed\n");  
    exit(EXIT_FAILURE);  
}  
  
/* Parent process */  
pid = wait(NULL);
```



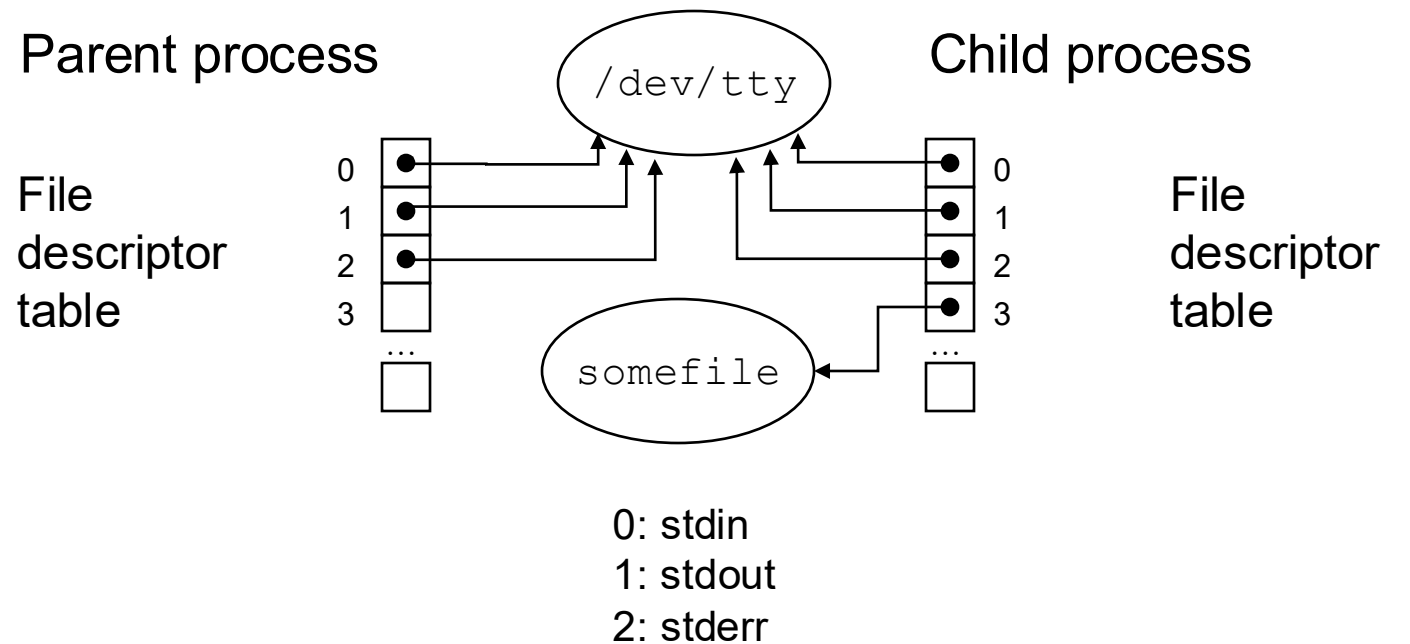
Simple Redirection Trace (2)

- Child process creates `somefile`

```
pid = fork();

if (pid == 0){
    /* Child process */
    fd = creat("somefile", 0640);
    dup2(fd, 1);
    close(fd);
    execvp("someprogram", program_args);
    fprintf(stderr, "exec failed\n");
    exit(EXIT_FAILURE);
}

/* Parent process */
pid = wait(NULL);
```



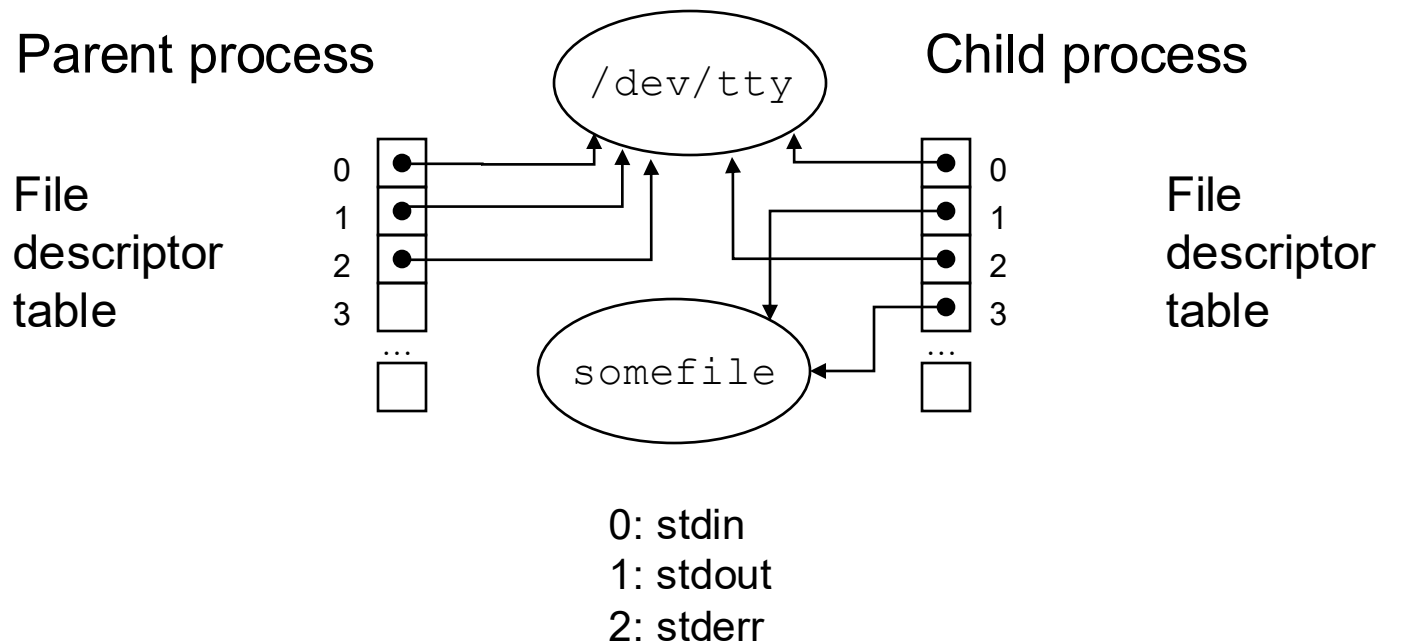
Simple Redirection Trace (3)

- `fd` and file descriptor number 1 can be used interchangeably after the successful call of `dup2(oldfd, newfd)`
 - duplicates the file descriptor `oldfd` onto `newfd`
 - `newfd` points the same open file description as `oldfd`.

```
pid = fork();

if (pid == 0){
    /* Child process */
    fd = creat("somefile", 0640);
    dup2(fd, 1);
    close(fd);
    execvp("someprogram", program_args);
    fprintf(stderr, "exec failed\n");
    exit(EXIT_FAILURE);
}

/* Parent process */
pid = wait(NULL);
```



Simple Redirection Trace (4)

- Closes `fd` (index 3 at file descriptor table)

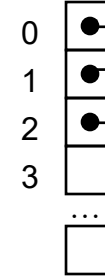
```
pid = fork();

if (pid == 0){
    /* Child process */
    fd = creat("somefile", 0640);
    dup2(fd, 1);
    close(fd);
    execvp("someprogram", program_args);
    fprintf(stderr, "exec failed\n");
    exit(EXIT_FAILURE);
}

/* Parent process */
pid = wait(NULL);
```

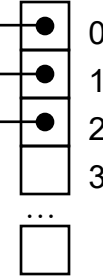
Parent process

File
descriptor
table



Child process

File
descriptor
table



/dev/tty

somefile

0: stdin
1: stdout
2: stderr

Simple Redirection Trace (5)

- someprogram **executes with** stdout **redirected to** somefile

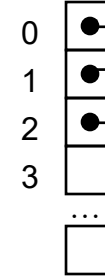
```
pid = fork();

if (pid == 0){
    /* Child process */
    fd = creat("somefile", 0640);
    dup2(fd, 1);
    close(fd);
    execvp("someprogram", program_args);
    fprintf(stderr, "exec failed\n");
    exit(EXIT_FAILURE);
}

/* Parent process */
pid = wait(NULL);
```

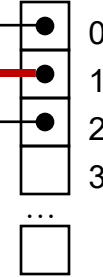
Parent process

File
descriptor
table



Child process

File
descriptor
table



somefile

Your T0-D0s for snush: Redirection

Functions for Redirection

- You need to implement `redout_handler()`
- Sample implementation of `redin_handler()` is provided in the skeleton code
- Important system calls to utilize
 - `open()`
 - `close()`
 - `dup2()`

Implementing Redirection in snush

- Redirect its input and output to a specified file before the command is executed
- `stdin` redirection: opens a file (w/ read-only perm.), replaces `stdin` with the fd
- `stdout` redirection: replaces `stdout` with the redirected fd
 - Set appropriate permissions for the output file
- **No need to support `stderr` redirection**

Support for Pipe

Example Use of Pipes

- The file “student_ID.txt” contains many student entries
- Find all students with the last name “Lee”
 - `$ grep Lee student_ID.txt`
- List the results in order of student id
 - `$ grep Lee student_ID.txt | sort -n`

```
booooong@600n9:~$ cat student_ID.txt
202600001 Sechang Kim
202600002 Sinu Park
202600010 Soowon Lee
202600006 Seungjoo Lee
booooong@600n9:~$ grep Lee student_ID.txt
202600010 Soowon Lee
202600006 Seungjoo Lee
booooong@600n9:~$ grep Lee student_ID.txt | sort -n
202600006 Seungjoo Lee
202600010 Soowon Lee
booooong@600n9:~$
```

System calls for Pipe

- A pipe allows two processes on the same machine to exchange data
 - `stdout` of the previous program flows into `stdin` of the next program
- How?
 - Use `pipe()` to create a pair of fds (2 fds) that are connected to each other
 - `pipe()` creates a unidirectional communication line (one for read-only, the other for write-only)
 - Use `fork()` to create a child process that shares the two fds with the parent
 - Use `dup2()` to connect the pipe fds to the `stdin`, `stdout` of the processes
- Pipe descriptors are file descriptors in UNIX

Unidirectional Communication

- `int pipe(int fd[2])` creates a unidirectional communication line
 - **`fd[0]` for read-end, `fd[1]` for write-end** are added to the table
 - Creates a pipe buffer inside the kernel
 - Usually used together with `fork()` to transfer data between a parent and child process.
 - When `pipe()` is called before `fork()`, both inherit file descriptors that refer to the same pipe object
 - To create one-way communication, each process closes the unused end of the pipe.

File descriptor table

0	1	2	fd[0]	fd[1]
---	---	---	-------	-------

0	1	2	fd[0]	fd[1]
---	---	---	-------	-------

Closed for unidirectional

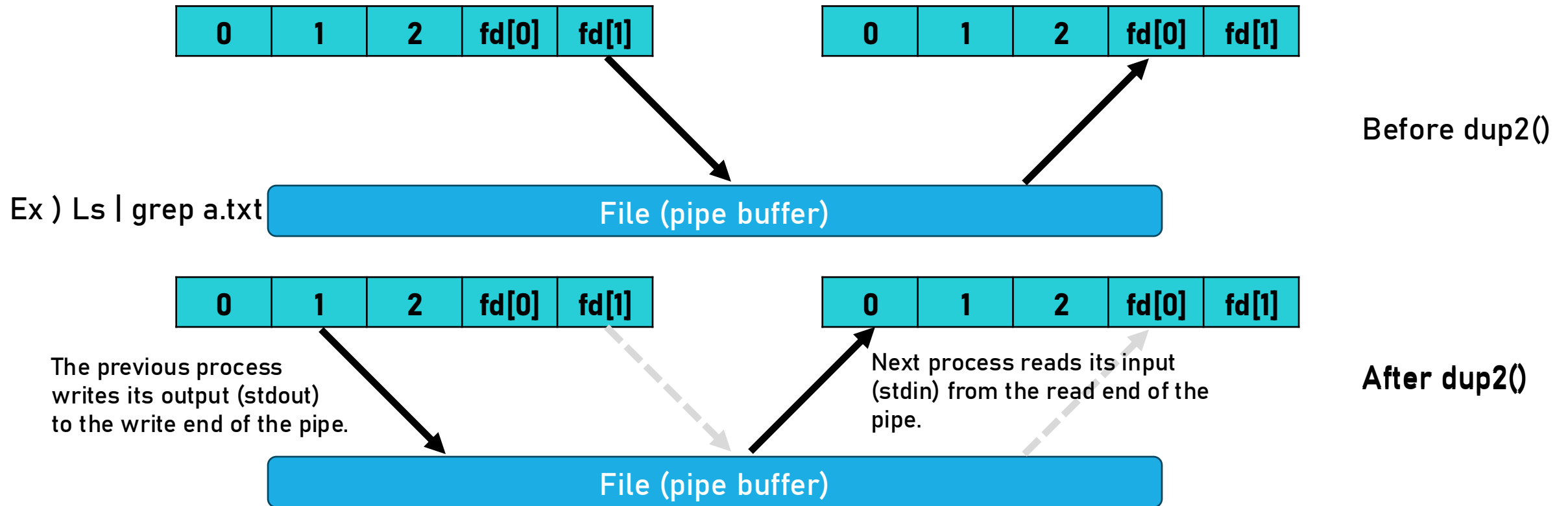
Closed for unidirectional

File (pipe buffer)

Duplicate to Other File Descriptors

- `dup2 ()`

- Use `dup2 ()` to duplicate the pipe ends onto `stdin`, `stdout`, or other file descriptors.
 - `dup2 (p[1], STDOUT_FILENO), dup2 (p[0], STDIN_FILENO);`
- The output of the process can be passed to the input of the next process



Simple Pipe example

- Example code
 - Inter Process Communication between the child and the parent

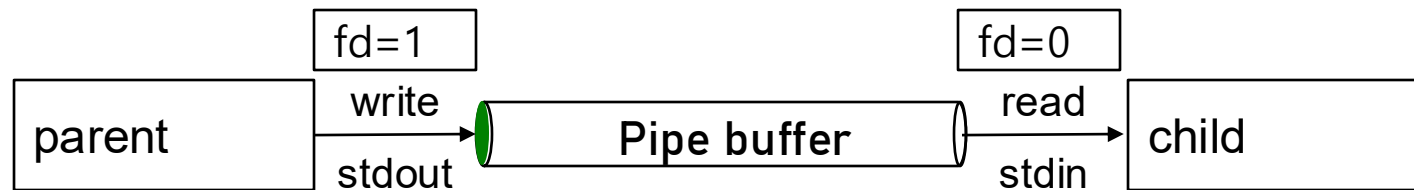
```
int pid, p[2];
if (pipe(p) == -1)
    exit(1);

pid = fork();
if (pid == 0) {
    close(p[1]);
    dup2(p[0], 0);
    close(p[0]);
    ... read from stdin ...
}
else {
    close(p[0]);
    dup2(p[1], 1);
    close(p[1]);
    ... write to stdout ...
    wait(&status);
}
```

① Create pipe read: fd[0],
write: fd[1]

② Child makes stdin (0) the
read side of the pipe

③ Parent makes stdout (1) the
write side of the pipe



**Your T0-D0 for snush:
Pipe**

Functions for Pipe

- You need to implement `iter_pipe_fork_exec()`
- Important system calls to utilize
 - `fork()`, `dup2()`, `execvp()`
 - `pipe()`, `close()`
 - `setpgid()`: Sets the process group ID of a process, allowing multiple processes to be grouped together for job control and signal management.
 - `tcsetpgrp()`
- Important `snush` functions to utilize (Do not modify the functions listed below)
 - `dynarray_get()`: get tokens of a specific index, where index starts at 0
 - `dynarray_get_length()`: get total number of tokens
 - `check_builtin()`, `execute_builtin_partial()`
 - `build_command_partial()`: construct the command array passed as argument to `execvp()`
 - `wait_fg()`: continuously checks whether the given job has finished executing

Implementing Pipe in snush

- Decompose the token array based on the pipe symbol (' | ')
 - Note that the pipe symbol cannot be preceded by an output ('>') or input ('<') redirection operator
- Set up pipes
- Create child processes and run each command on each child
 - Create a child process for each command in the pipe
 - Convert the tokenized input to an argument array for external command execution
 - Execute a specified command at the child process
 - **All processes that belong to a pipeline must have the same process group ID.**
- The output of the previous process becomes the input of the next process
- Use `wait_fg()` to make sure the parent process is waiting properly
- `iter_pipe_fork_exec()` should return the job id of the **child**

Tips for implementing pipeline in snush

- **All processes in a pipeline must share the same process group ID(PGID)**
 - A | B | C : A, B, C should have the same PGID
 - Use `setpgid()` to configure the process group
 - Be careful! — process scheduling after `fork()` is nondeterministic, so either the parent or the child may run first

Execution order of the shell and its child process

- In the `fork_exec()` & `iter_pipe_fork_exec()`
- A job must be added by the parent process before the child process begins executing the new binary program
- Use `pipe()`, `read()`, `write()` to ensure their ordering
 - By default, `read()` performs blocking I/O

Implementing Basic Shell Functions - `fork_exec()`

- Create a child process
- Convert the tokenized input into an argument array for an external command
- Do appropriate actions for supporting I/O redirection or a pipe
- Parent process should add job to a job manager
- Execute the command in the child process
- Make sure the parent process waits properly with `wait_fg()`
- The return value of `fork_exec()` should be the child's PID

Execution order matters!

Support for Background Execution

Example for Background Process Execution

- Run tasks in the background so you can keep using the shell
 - Can run other commands
- Useful for long-running processes (e.g., file downloads, backups, or large data processing)
- Child sends `SIGCHLD` signal to parent process when the child process exits

Simple Background Execution

- If parent process wants to wait for the child to finish, call `wait_fg()`
- Otherwise, do not wait for the child to finish its execution and call `print_job()`

```
pid = fork();

if (pid < 0) {
    perror("fork failed");
    exit(EXIT_FAILURE);
} else if (pid == 0) {
    execl("/bin/sleep", "sleep", "20", NULL);
    // execlp("sleep", "sleep", "20", NULL);

    perror("execlp failed");
    exit(EXIT_FAILURE);
} else {
    if (!background) {
        wait_fg();
    } else {
        print_job();
        //printf("Background: child process (PID: %d)\n", pid);
    }
    exit(EXIT_SUCCESS);
}
```

- ① Create a child process using `fork()`
- ② If it is a child process, change its execution image to another
- ③ If it is a parent process, decide whether to wait or not
- ④ If it needs a background execution, a parent process do not wait for the child to exit

Your T0-D0 for snush: Background Execution

Functions for Background Execution

- You need to implement/modify `check_sigchld()` & `check_bg_status()`
 - Also, `fork_exec()`, `iter_pipe_fork_exec()`
- `check_bg_status()` should print out the message that the background job has completed
- Important `snush` functions to utilize (Do not modify the functions listed below)
 - `print_job()`: announce the start of a background job

Implementing Background Execution in snush

- Child sends `SIGCHLD` signal to parent process when the child process exits
- Each exited process must be removed from the process id list of its corresponding job in the job manager
- A message should be printed at `check_bg_status()` to indicate that the background job has completed
 - Do not use `printf()` in signal handler (refer to the README file)
- The commands `fg` and `bg` are not supported in snush, so you do not need to consider them when implementing background jobs.
- In addition, the related signal `SIGTSTP` (triggered by Ctrl+Z) is set to be ignored in this implementation.

Final Guidelines for snush

Submission

- **Deadline: ~ 2026. 5. 29 23:59**
 - 0 Points if deadline is missed
 - 0 Points for copying
- Do not change the name and the prototype of the skeleton code
- Directory name should be `YourID_assign4` (ex. `202600000_assign4`)
- Place the entire source codes in a single directory `202600000_assign4`
- Make a gzipped tar file for submission
 - `tar zcf 202600000_assign4.tar.gz 202600000_assign4`
- Submit to **eTL** submission page

Out of bound of this shell lab

- **Stop and continuing of the job (`Ctrl + z`)**
 - Handling `SIGTST`
- **Handling `SIGPIPE` (Broken pipe handling - ignored by default)**
 - `$ yes | head -n 100` (does not stop until the `Ctrl + c` : understandable)
 - `$ yes | head -n 100` (Broken pipe error : understandable)
- **Handling `SIGTTOU`**
 - Terminal control on background process
 - Ignore the signal `SIGTTOU`

Grading

Sections	Points
Single command (built-in & external) (10)	100
I/O Redirection (10)	
Pipelines (20)	
Signals & Interrupt (10)	
Mixed Scenarios (50)	
Background process test (Extra credit)	30

- Notice

- Please read `README.md` thoroughly before asking questions. More detailed instructions are there.
- Please modify only the sections marked with **TODO** in the function.
- There is No Server for testing. Use `sample_snush` in `lab4_shell/` or `/bin/bash` as a reference for test comparison.
- Also, There will be some test cases will be included with files.
- Each grading section, such as pipelining, redirection, and in interrupts, contains 5 to 10 test cases. Points will be awarded for each test case that passes.
- However, for the extra-credit assignment, points are awarded only if **all test cases pass**.

THANK YOU
